



Chuyên đề: Thuật toán Duyệt Đồ thị: BFS & DFS (Graph Traversal & Search)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

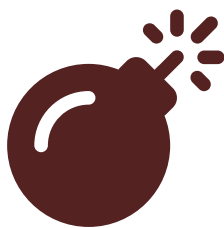
Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

🎯 1. Tổng quan & Bản đồ Tư duy (Mindmap)

Lý thuyết Đồ thị (Graph Theory) là một trong những trụ cột quan trọng nhất của Lập trình thi đấu (Competitive Programming). Bất kỳ hệ thống bài toán nào từ tìm đường, phân vùng mạng lưới, kiểm tra tính liên thông, lập lịch phụ thuộc đến cấu trúc cây đều khởi nguồn từ hai thuật toán duyệt đồ thị nền tảng:

- **Duyệt theo chiều rộng (BFS - Breadth-First Search):** Khám phá đồ thị theo từng "lớp sóng" (level-by-level) lan tỏa từ đỉnh nguồn bằng cấu trúc Hàng đợi (FIFO Queue). BFS là công cụ chuẩn mực để tìm đường đi ngắn nhất trên đồ thị không trọng số.
- **Duyệt theo chiều sâu (DFS - Depth-First Search):** Đâm sâu theo từng nhánh rẽ cho đến khi chạm đáy rồi quay lui (Backtrack) bằng Call Stack (Đệ quy) hoặc Ngăn xếp (LIFO Stack). DFS là nền tảng để xây dựng cây khung, phân loại cạnh, phát hiện chu trình, sắp xếp tô-pô và phục vụ các thuật toán nâng cao như Khớp-Cầu, Tarjan, 2-SAT.



Syntax error in text
mermaid version 10.9.8

📚 2. Biểu diễn Đồ thị Tối ưu trong C++

Trong lập trình thi đấu, có 3 cách biểu diễn đồ thị phổ biến:

2.1. Danh sách kề (Adjacency List) — *Khuyến dùng 95% trường hợp*

Sử dụng mảng các `std::vector` :

```
const int MAXN = 200005;
vector<int> adj[MAXN]; // Đồ thị không trọng số
// hoặc: vector<pair<int, int>> adj[MAXN]; // Đồ thị có trọng số (đỉnh kề, trọng số)

void add_edge(int u, int v, bool directed = false) {
    adj[u].push_back(v);
    if (!directed) {
        adj[v].push_back(u);
    }
}
```

- **Không gian bộ nhớ:** $O(V + E)$, tối ưu hoàn hảo cho đồ thị thưa ($E \ll V^2$).
- **Thời gian duyệt đỉnh kề:** $O(\deg(u))$.

2.2. Ma trận kề (Adjacency Matrix)

Sử dụng mảng hai chiều `bool adj[MAXN][MAXN]` hoặc `int cost[MAXN][MAXN]`:

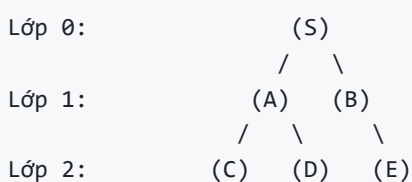
- **Áp dụng khi:** $V \leq 1000$ hoặc bài toán cần kiểm tra xem cạnh (u, v) có tồn tại hay không trong $O(1)$.
- **Hạn chế:** Tốn $O(V^2)$ bộ nhớ và $O(V^2)$ thời gian khởi tạo. Nếu $V = 10^5$, khai báo ma trận kề sẽ gây **Memory Limit Exceeded (MLE)** ngay lập tức!

3. Thuật toán Duyệt theo Chiều rộng (BFS)

3.1. Nguyên lý & Mô hình Lan tỏa

BFS xuất phát từ một đỉnh nguồn S , sử dụng một hàng đợi FIFO (`std::queue`) để quản lý các đỉnh chờ duyệt.

1. Khởi tạo hàng đợi `Q`, đẩy đỉnh nguồn S vào `Q`, gán khoảng cách `dist[S] = 0` và đánh dấu `visited[S] = true`.
2. Trong khi `Q` không rỗng:
 - Lấy đỉnh u ở đầu hàng đợi ra (`Q.front()` , `Q.pop()`).
 - Duyệt qua tất cả các đỉnh kề $v \in adj[u]$.
 - Nếu v chưa được thăm (`!visited[v]`):
 - Đánh dấu `visited[v] = true`.
 - Cập nhật khoảng cách: `dist[v] = dist[u] + 1`.
 - Lưu vết đường đi: `parent[v] = u`.
 - Đẩy v vào cuối hàng đợi: `Q.push(v)`.



Quy tắc sống còn trong BFS:

Phải gán `visited[v] = true` **NGAY KHI ĐẨY VÀO HÀNG ĐỢI** (`Q.push(v)`), **KHÔNG ĐƯỢC** đợi đến khi

lấy ra khỏi hàng đợi mới đánh dấu. Nếu đánh dấu muộn, cùng một đỉnh có thể bị đẩy vào hàng đợi hàng nghìn lần bởi các đỉnh kề khác nhau, dẫn đến TLE và MLE!

3.2. BFS Đa nguồn (Multi-Source BFS)

Bài toán: Cho đồ thị G và một tập các đỉnh nguồn $S = \{s_1, s_2, \dots, s_k\}$. Cần tìm khoảng cách ngắn nhất từ mỗi đỉnh $u \in V$ tới **đỉnh nguồn gần nhất** trong S .

- **Tiếp cận ngây thơ**: Chạy BFS k lần từ từng nguồn $\implies O(k \cdot (V + E)) \rightarrow$ TLE khi $k \approx V$.
- **Kỹ thuật Đa nguồn**: Đẩy **toàn bộ** các nguồn $s_i \in S$ vào hàng đợi ban đầu cùng lúc:

```
cpp queue<int> Q; for (int s : S) { dist[s] = 0; visited[s] = true; Q.push(s); }
```

Sau đó chạy vòng lặp BFS bình thường. Toàn bộ thuật toán chỉ mất đúng $O(V + E)$!

3.3. Thuật toán 0-1 BFS với `std::deque`

Khi đồ thị có trọng số cạnh chỉ nhận 1 trong 2 giá trị: 0 hoặc 1 (ví dụ: đổi hướng đi tốn 1, đi thẳng tốn 0):

- Thuật toán Dijkstra thông thường mất $O((V + E) \log V)$ với `priority_queue`.
- **0-1 BFS** giải quyết bài toán trong $O(V + E)$ **tuyến tính** bằng `std::deque`:
- Nếu đi qua cạnh trọng số 0: Khoảng cách không đổi $\implies$ đẩy v vào **ĐẦU** deque (`dq.push_front(v)`).
- Nếu đi qua cạnh trọng số 1: Khoảng cách tăng 1 $\implies$ đẩy v vào **CUỐI** deque (`dq.push_back(v)`).
- Tính chất hàng đợi luôn duy trì khoảng cách tăng đơn điệu không vượt quá 1 đơn vị chênh lệch giữa hai đầu!

3.4. Kiểm tra Đồ thị Hai phía (Bipartite Graph Check)

Một đồ thị là hai phía khi và chỉ khi tập đỉnh có thể phân hoạch thành 2 tập độc lập, tức là **có thể tô màu các đỉnh bằng 2 màu (0 và 1)** sao cho không có hai đỉnh kề nhau nào cùng màu.

- Dùng BFS tô màu:
- Gán `color[S] = 0`.
- Với mỗi đỉnh kề $v \in adj[u]$:
- Nếu v chưa tô màu: `color[v] = 1 - color[u]`, đẩy v vào queue.
- Nếu v đã tô màu và `color[v] == color[u]`: **Đồ thị không phải hai phía!**

📌 **Định lý**: Đồ thị là hai phía $\iff$ Đồ thị không chứa **chu trình có độ dài lẻ**.

🌲 4. Thuật toán Duyệt theo Chiều sâu (DFS)

4.1. Cơ chế Đệ quy & Cây khung DFS

DFS đi sâu vào một nhánh cho đến khi không thể đi tiếp được nữa, sau đó lùi lại đỉnh trước để khám phá nhánh khác:

```

void dfs(int u) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            parent[v] = u;
            dfs(v);
        }
    }
}

```

Trong quá trình duyệt đồ thị có hướng, các cạnh được phân thành 4 loại dựa trên cây khung DFS:

1. **Tree edge (Cạnh cây)**: Cạnh đi từ đỉnh cha u tới đỉnh con chưa thăm v trong cây DFS.
2. **Back edge (Cạnh ngược)**: Cạnh đi từ đỉnh u tới một **tổ tiên** của nó trong cây DFS.
 $\implies$ **Sự tồn tại của Back edge khẳng định đồ thị có chu trình!**
3. **Forward edge (Cạnh xuôi)**: Cạnh đi từ u tới một **con cháu** của nó (không phải cạnh cây trực tiếp).
4. **Cross edge (Cạnh chéo)**: Cạnh nối giữa hai nhánh cây khác nhau trong rừng DFS.

4.2. Thời gian Vào/Ra (tin / $tout$) & Mảng Thứ tự Euler

Ta duy trì một biến đếm toàn cục $timer = 0$:

- $tin[u]$: Thời điểm bắt đầu gọi $dfs(u)$ ($timer++$).
- $tout[u]$: Thời điểm kết thúc duyệt toàn bộ cây con gốc u ($timer++$).

```

int timer = 0;
int tin[MAXN], tout[MAXN];

void dfs(int u, int p = 0) {
    tin[u] = ++timer;
    for (int v : adj[u]) {
        if (v != p) {
            dfs(v, u);
        }
    }
    tout[u] = ++timer;
}

```

Định lý Quan hệ Tổ tiên trong $O(1)$:

Đỉnh u là tổ tiên của đỉnh v trên cây khi và chỉ khi:

$$tin[u] \leq tin[v] \quad \text{và} \quad tout[u] \geq tout[v]$$

Đoạn thời gian $[tin[u], tout[u]]$ hoàn toàn **chứa trọn vẹn** đoạn $[tin[v], tout[v]]$.

4.3. Phát hiện Chu trình bằng Kỹ thuật 3 Màu (Three-Color DFS)

Trên đồ thị có hướng, để phát hiện chu trình, ta phân loại mỗi đỉnh vào 1 trong 3 trạng thái màu:

- **Trắng (0 - Unvisited)**: Đỉnh chưa được thăm.
- **Xám (1 - Visiting / In Stack)**: Đỉnh đang được duyệt trong nhánh đệ quy hiện tại (nằm trong Call Stack).
- **Đen (2 - Visited / Completed)**: Đỉnh đã duyệt xong hoàn toàn toàn bộ cây con của nó.

```
int color[MAXN]; // 0: White, 1: Gray, 2: Black
bool has_cycle = false;

void dfs_cycle(int u) {
    color[u] = 1; // Tô xám: Đang duyệt
    for (int v : adj[u]) {
        if (color[v] == 0) {
            dfs_cycle(v);
        } else if (color[v] == 1) {
            // Gặp đỉnh Xám -> Tồn tại cạnh ngược (Back edge) -> Phát hiện chu trình!
            has_cycle = true;
        }
    }
    color[u] = 2; // Tô đen: Đã hoàn tất
}
```

4.4. Sắp xếp Tô-pô (Topological Sort)

Sắp xếp tô-pô của một đồ thị có hướng không chu trình (DAG - Directed Acyclic Graph) là một thứ tự tuyến tính của các đỉnh sao cho với mọi cạnh có hướng $u \rightarrow v$, đỉnh u luôn xuất hiện trước đỉnh v .

Cách 1: Bằng DFS (Thứ tự giảm dần của `tout`)

Mỗi khi duyệt xong một đỉnh (chuyển sang màu Đen), ta nạp đỉnh đó vào đầu danh sách kết quả (hoặc `push_back` rồi đảo ngược mảng):

```
vector<int> topo_order;
void dfs_topo(int u) {
    color[u] = 1;
    for (int v : adj[u]) {
        if (color[v] == 0) dfs_topo(v);
    }
    color[u] = 2;
    topo_order.push_back(u); // Nạp khi hoàn thành
}
// Sau khi duyệt tất cả các đỉnh: reverse(topo_order.begin(), topo_order.end());
```

Cách 2: Thuật toán Kahn (Bằng BFS và Bán bậc vào `in_degree`)

1. Tính bán bậc vào `in_degree[u]` cho mọi đỉnh.
2. Đẩy toàn bộ các đỉnh có `in_degree[u] == 0` vào hàng đợi.

3. Lấy u ra, ghi nhận vào thứ tự tô-pô. Với mỗi $v \in \text{adj}[u]$, giảm $\text{in_degree}[v]--$. Nếu $\text{in_degree}[v] == 0$, đẩy v vào hàng đợi.
4. Nếu số đỉnh lấy ra $< V$, đồ thị **chứa chu trình**!

🌀 5. Đồ thị Hàm (Functional Graphs / Successor Graphs)

Một đồ thị có hướng được gọi là **Đồ thị Hàm (Functional Graph)** nếu mọi đỉnh u đều có đúng một cạnh đi ra:

$$\text{out_degree}(u) = 1 \quad \forall u \in V$$

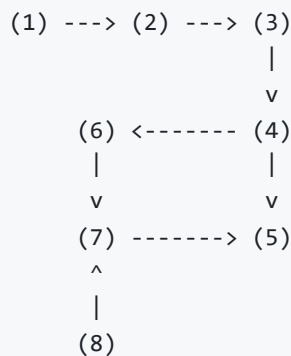
Mỗi đỉnh u có duy nhất một đỉnh kế tiếp: $f(u)$.

5.1. Cấu trúc Hình chữ Rho (ρ -shape)

Vì số đỉnh là hữu hạn (N) và mỗi đỉnh đều có cạnh đi ra, khi xuất phát từ bất kỳ đỉnh nào và đi theo hàm $f(u)$, theo nguyên lý Dirichlet, ta chắc chắn sẽ lặp lại một đỉnh đã từng qua.

Do đó, mỗi thành phần liên thông của Functional Graph luôn bao gồm:

1. Đúng **một chu trình** (Cycle) nằm ở trung tâm.
2. Một tập các **cây có hướng** (Directed Trees) mà các cạnh đều hướng về phía chu trình.



(Trong sơ đồ trên, các đỉnh 4, 5, 7, 6 tạo thành chu trình; các nhánh 1-2-3 và 8 đổ về chu trình).

5.2. Thuật toán Rùa & Thỏ (Floyd's Cycle-Finding Algorithm)

Để tìm chu trình trên Functional Graph với $O(1)$ bộ nhớ phụ:

- Con trỏ Rùa (`tortoise`): Đi 1 bước mỗi lần: $t \leftarrow f(t)$.
- Con trỏ Thỏ (`hare`): Đi 2 bước mỗi lần: $h \leftarrow f(f(h))$.
- Hai con trỏ chắc chắn gặp nhau bên trong chu trình!
- Sau khi gặp nhau, đặt lại một con trỏ về đỉnh xuất phát và cho cả hai đi cùng vận tốc 1 bước: vị trí gặp nhau tiếp theo chính là **đỉnh đầu tiên bước vào chu trình**!

6. Bảng So sánh Toàn diện: BFS vs DFS

Tiêu chí	Duyệt theo Chiều rộng (BFS)	Duyệt theo Chiều sâu (DFS)
Cấu trúc Dữ liệu	Hàng đợi FIFO (<code>std::queue</code> , <code>std::deque</code>)	Ngăn xếp LIFO / Call Stack (Đệ quy)
Độ phức tạp Thời gian	$O(V + E)$	$O(V + E)$
Độ phức tạp Bộ nhớ	$O(V)$ (đỉnh trên cùng một tầng)	$O(H)$ (H là độ sâu cây đệ quy)
Đường đi ngắn nhất	Tối ưu trên đồ thị không trọng số	Không đảm bảo ngắn nhất
Phát hiện chu trình	Tốt trên đồ thị vô hướng	Xuất sắc trên cả có hướng (3 màu) và vô hướng
Sắp xếp Tô-pô	Thuật toán Kahn (Bán bậc vào)	Đảo ngược thời điểm thoát <code>tout</code>
Ứng dụng đặc thù	0-1 BFS, Multi-source BFS, Bipartite check	Khớp-Cầu, Thành phần liên thông mạnh (SCC), DP trên cây

7. Cạm bẫy Phòng thi & Sai lầm Thường gặp (Pitfalls)

- Tràn Call Stack (Stack Overflow) khi DFS cây suy biến:**
- Trên hệ điều hành Windows hoặc trong môi trường thi đấu, kích thước ngăn xếp mặc định có thể chỉ là 8MB. Khi cây có dạng đường thẳng ($N = 2 \cdot 10^5$), `dfs()` đệ quy sâu $2 \cdot 10^5$ tầng sẽ làm chương trình bị **Crash (Runtime Error)**!
- Khắc phục:** Tăng kích thước ngăn xếp khi biên dịch (`-Wl,--stack=268435456`) hoặc chuyển sang DFS khử đệ quy bằng `std::vector` mô phỏng stack.
- Đánh dấu `visited` muộn trong BFS:**
- Như đã nhấn mạnh: Luôn gán `visited[v] = true` ngay khi `push(v)` vào queue. Nếu đợi `pop()` mới đánh dấu, một đỉnh bậc cao sẽ bị nạp hàng trăm nghìn lần gây TLE/MLE.
- Quên vòng lặp quét đồ thị không liên thông:**
- Đồ thị có thể gồm nhiều thành phần rời rạc: Luôn chạy vòng lặp ngoài:

```
cpp for (int u = 1; u <= n; ++u) { if (!visited[u]) bfs(u); // hoặc dfs(u) }
```
- Lẫn lộn giữa chu trình vô hướng và có hướng:**
- Đồ thị vô hướng: Cạnh ngược về chính `parent` trực tiếp **không** phải là chu trình! Luôn truyền tham số `parent` để bỏ qua: `if (v == p) continue;`
- Đồ thị có hướng: Bắt buộc phải dùng 3 màu (Trắng/Xám/Đen) để tránh nhầm Cross edge thành chu trình.

8. Tài liệu Tham khảo & Đọc thêm

1. **Competitive Programmer's Handbook** — *Antti Laaksonen*:
 2. *Chapter 11: Basics of Graphs (BFS, DFS, Connectivity)*.
 3. *Chapter 16: Directed Graphs (Topological sort, Functional graphs)*.
 4. **Introduction to Algorithms (CLRS 4th Edition)**:
 5. *Chapter 20: Elementary Graph Algorithms (Breadth-first search, Depth-first search, Topological sort)*.
 6. **USACO Guide (Silver / Gold)**:
 7. [Graph Traversal \(DFS/BFS\)](#).
 8. [Topological Sort & Functional Graphs](#).
 9. **VNOI Wiki**:
 10. [Lý thuyết Đồ thị Cơ bản](#).
 11. [Các thuật toán tìm kiếm trên đồ thị \(BFS & DFS\)](#).
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.